

Anatomía Paso a Paso de Código PHP y Patrón MVC

Explicación exhaustiva, signo por signo, palabra por palabra, función por función y línea por línea del proyecto. Diseñado especialmente para enseñar programación sin dejar dudas a los estudiantes.

1

El Diccionario de Signos y Símbolos de PHP

Cuando los estudiantes inician en PHP, los símbolos suelen ser la mayor fuente de confusión. Esta tabla desglosa cada signo que aparece en el proyecto con su pronunciación, significado y función exacta.

Sígnos	Nombre Didáctico	¿Para qué sirve exactamente?	Ejemplo en el Proyecto
	Etiquetas de Apertura y Cierre	Le ordenan al servidor web: <i>"A partir de aquí interpreta el texto como código PHP, no como simple texto HTML"</i> .	<code><?php class Usuario { ... } ?></code>
	Símbolo de Dólar (Variable)	Identificador obligatorio de variables. Le indica a PHP que la siguiente palabra es un contenedor en memoria para guardar datos.	<code>\$email, \$nombre, \$pdo</code>
	Flecha de Objeto / Instancia	Accede a una propiedad o método de un objeto que ya fue instanciado en memoria (<code>\$objeto->metodo()</code>).	<code>\$pdo->prepare(...)</code> <code>\$consulta->execute()</code>
	Doble Dos Puntos (Paamayim Nekudotayim)	Operador de resolución de ámbito. Accede a constantes o métodos estáticos de una clase sin instanciarla (sin usar new).	<code>baseDatos::crearInstancia()</code> <code>PDO::PARAM_STR</code>
	Flecha Doble (Asociación Clave-Valor)	En los arrays asociativos , asigna un valor a una clave específica (<code>"clave" => "valor"</code>).	<code>PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION</code>
	Asignación vs Comparación	<code>=</code> guarda un dato en una variable. <code>==</code> compara si dos valores son iguales. <code>>=</code> compara si es mayor o igual.	<code>\$clave = \$_POST["..."]</code> (Guarda) <code>if (\$numeroRegistros >= 1)</code> (Compara)
	Punto y Coma	Finalizador de instrucción. Le dice a PHP: <i>"Aquí termina esta orden, pasa a la siguiente"</i> . Olvidarlo provoca el famoso Parse error.	<code>\$nombre = "Juan";</code>

Signo	Nombre Didáctico	¿Para qué sirve exactamente?	Ejemplo en el Proyecto
.	Punto de Concatenación	Une (pega) dos cadenas de texto o una variable con un texto.	<code>header("Location: " . URL_BASE)</code>
[]	Corchetes de Array	1. Definen un arreglo: [1, 2, 3]. 2. Acceden a la posición o clave de un arreglo: <code>\$_POST["correo"]</code> .	<code>\$_POST["txtEmail"]</code> <code>\$url[0]</code>
{ }	Llaves de Bloque	Delimitan el cuerpo de clases, funciones, bucles y condicionales (donde empieza y termina el bloque de código).	<code>class Usuario { ... }</code> <code>if(...) { ... }</code>
()	Paréntesis de Parámetros/Condición	1. Reciben argumentos en funciones: <code>crear(\$a, \$b)</code> . 2. Encierran condiciones en <code>if(...)</code> .	<code>if(isset(\$_POST["login"]))</code>
!	Signo de Exclamación (Negación Lógica)	Invierte el valor booleano. "NO". <code>!isset(\$x)</code> significa "Si NO está definida la variable \$x".	<code>if (!isset(self::\$instancia))</code>
:parametro	Dos Puntos en SQL (Marcador de Posición)	Marcador de parámetro nombrado en PDO para evitar Inyecciones SQL. Evita concatenar variables directamente en la consulta.	<code>WHERE correo = :correo</code>

2 Palabras Clave y Superglobales Fundamentales

`class` & `function`

class: Es el molde o plano para crear objetos. Agrupa variables (propiedades) y acciones (métodos).

function: Define un bloque de código ejecutable que realiza una tarea específica y puede retornar un resultado.

`public` vs `private`

public: El método o atributo puede ser accedido y usado desde cualquier parte del programa.

private: Sólo puede ser usado *dentro* de la misma clase. Oculta detalles internos (Encapsulamiento).

`static` & `self::`

static: Pertenecer a la clase en sí y no a un objeto individual. No requiere `new`.

self::: Es la forma en que una clase se refiere a sí misma para acceder a sus miembros estáticos.

`try` & `catch`

Estructura para manejar errores graves de forma segura. `try` intenta ejecutar código riesgoso (ej. conectar a la BD) y `catch` atrapa el error si falla sin tumbar la aplicación.

`include_once`

Inserta y ejecuta otro archivo PHP dentro del actual. El sufijo `_once` previene errores asegurando que no se incluya dos veces el mismo archivo.

`die()`

Detiene la ejecución del script PHP de inmediato e imprime un mensaje en pantalla. Se usa en errores críticos irreversibles.

Las Variables Superglobales de PHP

¿Qué es una Superglobal? Son arrays automáticos creados por PHP que están disponibles en cualquier parte del código, sin necesidad de declararlos con `global`.

- **`$_POST`** : Recibe datos enviados de forma oculta desde formularios HTML con método `method="POST"` (ej. contraseñas, correos).
- **`$_GET`** : Recibe datos visibles a través de la barra de direcciones URL (ej. `?url=productos/crear&id=5`).
- **`$_SESSION`** : Almacena información persistente del usuario (como su sesión iniciada) mientras navega por diferentes páginas del sitio.

Este controlador gestiona las peticiones del usuario relacionadas con el registro y la autenticación (Login). Veamos cada línea en detalle.

```
1 <?php
2 include_once "conexion.php";
3 include_once "../modelos/usuarios.php";
4
5 class ControladorUsuarios {
```

Líneas 1 a 5: Importaciones y Cabecera de Clase

- **Línea 1 (<?php):** Abre la sección de código ejecutable de PHP.
- **Línea 2 (include_once "conexion.php");:** `include_once` busca el archivo `conexion.php` y lo carga en memoria. El punto y coma `;` termina la orden. Permite que este controlador pueda usar la clase `baseDatos`.
- **Línea 3 (include_once "../modelos/usuarios.php");:** Importa el modelo `Usuario` ubicado en la carpeta `modelos/`. El punto `.` representa la carpeta actual.
- **Línea 5 (class ControladorUsuarios {}):** `class` declara una nueva clase.

`ControladorUsuarios` es el nombre de la clase (en convención *PascalCase*). La llave `{` abre el bloque donde se definirán todos sus métodos (acciones).

Método: crear() (Registro de Nuevos Usuarios)

```
7  public function crear(){
8  if($_POST){
9  $perfil = $_POST["txtperfil"];
10 $nombre = $_POST["txtnombre"];
11 $correo = $_POST["txtEmail"];
12 $clave = $_POST["txtPassword"];
13 Usuario::crear($perfil, $nombre, $correo, $clave);
14 header("Location: " . URL_BASE . "paginas/login");
15 }
16 }
```

Explicación Paso a Paso de crear()

- **Línea 7 (public function crear()):** **public** significa accesible desde afuera; **function** define la función llamada crear; los paréntesis () indican que no recibe argumentos directos al invocarse.
- **Línea 8 (if(\$_POST){):** Evalúa si el arreglo superglobal **\$_POST** contiene información (es decir, si el usuario presionó el botón de enviar del formulario). Si está vacío, se evalúa como falso y no ejecuta el bloque.
- **Líneas 9 a 12 (\$perfil=\$_POST["txtperfil"]; ...):** Extraen los valores de los campos de texto del formulario HTML mediante sus atributos name ("txtperfil", "txtnombre", "txtEmail", "txtPassword") y los guardan en variables limpias: \$perfil, \$nombre, \$correo, \$clave.
- **Línea 13 (Usuario::crear(\$perfil, \$nombre, \$correo, \$clave);):** **Usuario** es la clase del modelo; los dos puntos dobles **::** indican llamada estática; **crear(...)** pasa los 4 datos al modelo para que los inserte en la base de datos SQL.
- **Línea 14 (header("Location: " . URL_BASE . "paginas/login");):** **header()** envía una cabecera HTTP al navegador; Location: le ordena al navegador redirigir al usuario; los puntos **.** concatenan la constante URL_BASE con la ruta "paginas/login".

Método: ingreso() (Autenticación y Consulta PDO)

```
18 public function ingreso(){
19     if(isset($_POST["login"])){
20         include_once("../conexion.php");
21         $email = $_POST["txtEmail"];
22         $password = $_POST["txtPassword"];
23         $pdo = baseDatos::crearInstancia();
24         $consulta = $pdo->prepare("SELECT * FROM usuarios WHERE correo=:correo AND
clave=:clave");
25         $consulta->bindParam("correo", $email, PDO::PARAM_STR);
26         $consulta->bindParam("clave", $password, PDO::PARAM_STR);
27         $consulta->execute();
28         $registroEncontrado = $consulta->fetch(PDO::FETCH_ASSOC);
29         $numeroRegistros = $consulta->rowCount();
30         if ($numeroRegistros ≥ 1){
31             $_SESSION["usuario"] = $registroEncontrado;
32             header("Location: " . URL_BASE);
33         }else{
34             echo "USUARIO O CONTRASEÑA INVALIDOS";
35         }
36     }
37 }
```

Explicación Detallada de Cada Palabra y Función en ingreso()

- Línea 19 (if(isset(\$_POST["login"]))): **isset()** es una función nativa que comprueba si una variable *está definida y no es nula*. Comprueba si el botón de nombre login fue presionado.
- Línea 23 (\$pdo = baseDatos::crearInstancia());: Llama al método estático **crearInstancia()** de la clase baseDatos y guarda el objeto de conexión activa en la variable **\$pdo**.
- Línea 24 (\$consulta = \$pdo->prepare("SELECT * FROM usuarios WHERE correo=:correo AND clave=:clave"));: El operador **->** llama al método **prepare()** del objeto PDO. Envía la consulta SQL al motor de base de datos con los marcadores de posición :correo y :clave. Esto compila la consulta y **previene ataques de inyección SQL**.
- Líneas 25 y 26 (\$consulta->bindParam("correo", \$email, PDO::PARAM_STR));: **bindParam()** vincula la variable PHP \$email con el marcador :correo. La constante **PDO::PARAM_STR** le indica a la base de datos que este valor debe ser tratado estrictamente como una cadena de texto (String), impidiendo que se inyecte código malicioso.
- Línea 27 (\$consulta->execute());: Ejecuta la consulta preparada en la base de datos con los parámetros ya sustituidos de forma segura.
- Línea 28 (\$registroEncontrado = \$consulta->fetch(PDO::FETCH_ASSOC));: **fetch()** extrae la fila resultante de la base de datos. **PDO::FETCH_ASSOC** le indica que devuelva un arreglo asociativo donde las claves sean los nombres de las columnas (ej: \$registroEncontrado["nombre"]).
- Línea 29 (\$numeroRegistros = \$consulta->rowCount());: **rowCount()** cuenta cuántas filas coincidieron con la búsqueda. Si las credenciales son correctas, retornará 1.

- **Líneas 30 a 32** (`if ($numeroRegistros >= 1){ $_SESSION["usuario"] = ... }`): Si se encontró al menos un registro (`>= 1`), guarda los datos del usuario en la sesión del navegador `$_SESSION["usuario"]` y lo redirige a la página principal con `header()`.
- **Líneas 33 a 35** (`else { echo "USUARIO O CONTRASEÑA INVALIDOS"; }`): Si no coincide ninguna fila, `echo` imprime en pantalla el mensaje de error.

4

Análisis de Conexión a Base de Datos y Singleton: `conexion.php`

Este archivo implementa el **Patrón de Diseño Singleton**, el cual garantiza que solo exista *una única conexión* a la base de datos abierta durante toda la ejecución de la página, ahorrando memoria y conexiones del servidor MySQL.

```
3  class baseDatos {
5  private static $instancia = null;
11 public static function crearInstancia() {
13     if (!isset(self::$instancia)) {
23         $dsn = "mysql:host={$servidor};dbname={$baseDatos};charset={$charset}";
35         try {
37             self::$instancia = new PDO($dsn, $usuario, $password, $opcionesPDO);
38         } catch (PDOException $error) {
40             die("Error al conectar a la base de datos: " . $error->getMessage());
41         }
42     }
45     return self::$instancia;
46 }
47 }
```

Desglose de Conceptos Clave de `conexion.php`

- **Línea 5** (`private static $instancia = null;`): **private** impide que cualquiera fuera de la clase modifique la variable directamente. **static** hace que la variable pertenezca a la clase y no se reinicie entre llamadas. Inicializa en **null** (vacía).
- **Línea 13** (`if (!isset(self::$instancia)) {}`): El signo **!** niega la condición. **self::** accede a la propiedad estática. Significa: *"Si la instancia TODAVÍA NO existe, entra a crearla"*. Si ya existía, se salta todo y la devuelve de inmediato (Línea 45).
- **Línea 23** (`$dsn = "mysql:host={$servidor};dbname={$baseDatos};charset={$charset}";`): **DSN** significa *Data Source Name* (Nombre de Origen de Datos). Es el texto estructurado con el que PDO sabe a qué motor conectarse (mysql), en qué servidor (localhost), a qué base de datos y con qué juego de caracteres (utf8mb4 para tildes y eñes).
- **Línea 37** (`self::$instancia = new PDO($dsn, $usuario, $password, $opcionesPDO);`): **new** instancia un nuevo objeto de la clase nativa **PDO** (PHP Data Objects) pasándole los 4 parámetros de configuración y lo almacena en la variable estática.
- **Línea 38 a 40** (`catch (PDOException $error) { die(...) }`): Si la contraseña de MySQL estuviera mal o el servidor apagado, se genera un objeto de excepción **PDOException** guardado en `$error`. Con **→** `getMessage()` extrae la causa del fallo y **die()** detiene el script con un mensaje claro.

5

El Núcleo de la Arquitectura MVC: `index.php` y `ruteador.php`

El patrón MVC en PHP utiliza un **Front Controller** (Controlador Frontal). Todas las peticiones de los usuarios entran por `index.php`, el cual analiza la URL y delega la ejecución al `ruteador.php`.



Análisis de `ruteador.php`

```
1 <?php
2 include_once "controladores/controlador_{$getcontrolador}.php";
3 $controlador = "Controlador" . ucfirst($getcontrolador);
4 $objcontrolador = new $controlador();
5 $objcontrolador->$getaccion();
6 ?>
```

Magia del Ruteo Dinámico (Línea por Línea)

- **Línea 2** (`include_once "controladores/controlador_{$getcontrolador}.php";`): PHP reemplaza dinámicamente la variable `$getcontrolador` dentro de las comillas dobles. Si la variable vale "usuarios", la ruta resultante es "controladores/controlador_usuarios.php".
- **Línea 3** (`$controlador = "Controlador" . ucfirst($getcontrolador);`): **`ucfirst()`** (Uppercase First) convierte la primera letra en mayúscula. Si `$getcontrolador` es "usuarios", `ucfirst("usuarios")` devuelve "Usuarios". Al concatenar con "Controlador", genera el nombre exacto de la clase: "ControladorUsuarios".
- **Línea 4** (`$objcontrolador = new $controlador();`): **Instanciación Dinámica**: PHP permite usar una variable que contenga un string como nombre de clase al usar `new`. Equivale a escribir `new ControladorUsuarios()`.
- **Línea 5** (`$objcontrolador->$getaccion();`): **Invocación Dinámica de Método**: Llama al método cuyo nombre está guardado en la variable `$getaccion`. Si `$getaccion` vale "crear", ejecuta `$objcontrolador->crear()`.

? ¿Por qué a veces usamos -> y otras veces ::?

Regla de oro para los alumnos:

- Usa **->** cuando tengas un **objeto vivo** creado con new (ej: `$pdo->prepare()`, `$consulta->execute()`).
- Usa **::** cuando quieras llamar a una **clase directamente** o a una constante sin crear un objeto (ej: `baseDatos::crearInstancia()`, `PDO::PARAM_STR`).

? ¿Por qué no escribir directamente `SELECT * FROM usuarios WHERE correo = '$email'`?

Respuesta de Seguridad (Inyección SQL): Si concatenamos variables directamente en SQL, un usuario malicioso podría escribir en el formulario `' OR '1'='1` y entrar al sistema sin contraseña.

Al usar **`prepare()`** y **`bindParam()`**, el motor SQL separa la orden de los datos, haciendo imposible que el texto introducido sea interpretado como comandos SQL.

? ¿Qué diferencia hay entre `include`, `require` e `include_once`?

Instrucción	Si el archivo no existe...	¿Permite duplicados?
<code>include</code>	Muestra advertencia (Warning) y el script continúa.	Sí (puede redefinir clases y dar error).
<code>require</code>	Lanza error fatal y detiene el script.	Sí.
<code>include_once</code>	Muestra advertencia pero sólo lo carga una vez .	No (ignora si ya fue cargado).

Consejo para la clase: Invita a tus alumnos a revisar los errores en el archivo `php_error.log` de Laragon cada vez que vean una pantalla blanca o un error 500. El 90% de los errores de principiante son:

1. Olvidar el punto y coma **`;`** al final de una línea.
2. Escribir mal el nombre de una variable (ej: `$correo` vs `$email`).
3. No abrir la sesión con `session_start()` antes de usar `$_SESSION`.